Zero-Touch Federated Learning Attacks with an LLM-Directed Reinforcement-Learning Attacker

Pasindu Mallawarachchi*, Isuru Madusanke†, Tharushika Nettasinghe‡, Dinuth Kumara§,
Chamara Sandeepa¶, Tharindu Gamage‖, Madhusanka Liyanage**
*†‡§‖Faculty of Engineering, University of Ruhuna, Sri Lanka
¶**Network Softwarization and Security Labs (NetsLab), School of Computer Science, University College Dublin, Ireland
*mallawarachchi_pm_e22@engug.ruh.ac.lk, †madusanke_im_e22@engug.ruh.ac.lk, ‡nettasinghe_tn_e22@engug.ruh.ac.lk,
§kumara_dk_e22@engug.ruh.ac.lk
¶abeysinghe.sandeepa@ucdconnect.ie, ‖tharindu@eie.ruh.ac.lk, **madhusanka@ucd.ie

***Abstract*—Federated learning (FL) aggregates client model updates without inspecting raw client data, so a central server has no direct way to verify that a submitted update reflects honest training. We present a zero-touch FL attacker: a large language model (LLM) policy, trained with Group-Relative Policy Optimization (GRPO), that autonomously decides which of its controllable clients to poison each round and composes an ordered plan of primitive weight-space operators for each. A deterministic interpreter, not the LLM, applies the plan to a client's honest update, so the policy can never edit raw model weights directly. Training uses a single verifiable, continuous reward that jointly credits induced accuracy loss and evasion of a training-time detector, while penalizing malformed output, unnecessary client use, and near-duplicate (Sybil-like) collusion when several clients are used jointly. We benchmark the frozen, trained attacker by replaying its committed action, unmodified, against four independently evolving, fixed server-side defenses drawn from the published literature, FLTrust, Multi-Krum, DnC, and DeFL, none of which is a contribution of this paper and none of which the attacker adapts to at evaluation time; they serve purely as external benchmarks to probe how far the learned attack transfers. Across two poison-budget regimes, a strict minority of controllable clients and a coalition spanning half the federation, the learned attacker is fully neutralized by Multi-Krum and FLTrust at the small budget, and, at the larger budget, drives DeFL and DnC to near-total attack success while FLTrust still holds the global model close to its clean baseline. Per-round detection metrics did not reliably predict which defense actually protected the global model. These are preliminary, single-seed results and should be read as evidence of the mechanism's behavior rather than a fully replicated finding.**

***Index Terms*—federated learning, model poisoning, large language models, reinforcement learning, Group-Relative Policy Optimization, Byzantine-robust aggregation, adversarial machine learning, autonomous attackers**

I. Introduction

Federated learning (FL) lets edge devices collaboratively train a shared model without exposing local data [1]. Each round, a central server broadcasts the global model, every client trains locally, and the server aggregates the returned updates, typically by Federated Averaging (FedAvg), into a new global model. Because the server sees only model updates, it has no independent way to check that a given update reflects honest training, and a subset of compromised clients can submit manipulated updates that degrade the global model's accuracy [2], [3]. Existing attack studies almost always hand-design the attacker: a fixed rule chooses which clients to corrupt and a fixed formula perturbs their updates [10]–[12]. Such rules are cheap to defend against once known, and a new defense typically forces a human to redesign the attack.

We ask instead whether an attacker can be learned end-to-end from interaction. An LLM policy observes only dimensionless statistics of its own controllable clients' honest updates, no cross-client reference and no defense internals, and must decide, every round, which clients to poison and how. Its output is a small domain-specific-language (DSL) plan of primitive weight operators, applied by a deterministic interpreter so the policy never touches raw weights directly. We train this policy with GRPO [15] against a single verifiable reward that combines accuracy degradation, evasion of a training-time detector, output validity, client-budget efficiency, and diversity across jointly poisoned clients.

To measure whether the resulting strategy generalizes, we freeze the trained attacker and replay its committed action, unmodified, against a panel of four fixed, published server-side defenses that each evolve their own global model independently. None of these defenses is designed in this paper; they are used purely as external benchmarks to test the attacker. This isolates the defense as the only variable in the comparison and tests transfer to defenses the attacker never trained against.

Contributions. This paper makes the following contributions:

- A zero-touch FL attacker in which client selection and weight-operator composition are generated directly by an LLM policy, over a general-purpose primitive-operator DSL, without prescribing any attack strategy in advance.
- A verifiable, continuous reward that jointly credits accuracy degradation and evasion while penalizing malformed output, the use of more controllable clients than necessary, and non-diverse (Sybil-like) multi-client collusion, computed from ground truth available only during training.
- A GRPO training recipe with stochastic opponent scoring and an opponent league that keeps the attacker's learning signal alive against an evolving, non-deterministic detector, avoiding the zero-advantage collapse that a small, near-binary action space is prone to.
- A *fixed-attack, varied-defense benchmark harness* that replays one committed attack plan per round against four independently evolving, published defenses across two poison-budget regimes, testing whether the learned evasion strategy transfers to defenses never seen during training.
- An open system design covering the attacker contract, reward, training schedule, and benchmark protocol, forming a reusable testbed for studying autonomous, LLM-directed FL attacks.

The remaining sections cover related work, the threat model, the attack mechanism, GRPO training, the experimental setup, results, limitations, and future work.

II. Related Work

A. Federated Learning and the Poisoning Attack Surface

In a standard FL round, a central server distributes the global model to N clients, each performs local training, and the server aggregates

the returned updates through FedAvg [1]. Model poisoning occurs when compromised clients submit manipulated updates. Untargeted attacks degrade overall accuracy, whereas targeted backdoor attacks introduce a narrower vulnerability while leaving overall accuracy largely intact [2], [13]. This paper focuses on reducing the global model's overall test accuracy while evading detection. Non-IID data partitioning makes detection harder because an honest client's update can legitimately differ from the majority.

B. Byzantine-Robust and Statistical Defenses

Coordinate-wise median and trimmed-mean aggregation bound the influence of individual coordinates [4]. Krum and Multi-Krum retain updates that are close to their neighbors under Euclidean distance [5]. DnC centers the updates, projects them onto the leading right singular vector, and removes clients with large projections [6]. FLTrust derives a trusted direction from a clean server dataset and forms a cosine-weighted average [7]. FoolsGold reduces the weight of clients with similar contribution histories [8]. DeFL tracks layer-wise gradient norms and applies an adaptive outlier vote during critical learning periods [9]. FLAME clusters client updates and combines adaptive clipping with noise injection to remove backdoor contributions while bounding accuracy loss [20]. Each method relies on a fixed geometric or statistical prior; Section V-E uses four such defenses, none of which the attacker studied here trains against, purely as evaluation targets.

C. Adaptive and Learning-Based Attacks

A defense-aware attacker can construct updates that satisfy a robust aggregator's assumptions while still corrupting the result. Fang et al. optimize local model poisoning against known aggregation rules [10]. Bhagoji et al. show that one persistent attacker using boosted updates can bias the global model [3]. Xie et al. manipulate the inner product between the aggregate and the true gradient to slip past distance-based screens [11], and Baruch et al. show that small, carefully bounded perturbations from several colluding clients ("a little is enough") can evade defenses tuned to catch large outliers [12]. Wang et al. target rare data subpopulations to plant durable backdoors [13], and Cao and Gong fabricate fake client identities entirely (an orthogonal, Sybil attack surface we do not model here) [14]. Unlike this body of work, our attacker receives no closed-form knowledge of the aggregation rule and no white-box access to defense parameters, thresholds, or scores; it learns a policy purely from interaction outcomes.

D. Reinforcement Learning of LLM Policies

GRPO samples G completions per state, assigns each a verifiable reward, and normalizes rewards within the group to obtain the policy advantage without a separate critic network [15], building on the clipped policy-gradient framework of Proximal Policy Optimization (PPO) [16] and on RLHF-style fine-tuning of instruction-following LLMs [17]. GRPO fits our setting because each attack round produces one terminal, exactly computable reward. Low-Rank Adaptation (LoRA) provides a small trainable adapter over one shared, frozen base model [18]; we use the open-weight Llama-3.2 family as that base [19].

III. Threat Model

Problem statement. Existing FL security studies commonly evaluate a fixed, hand-designed attack against a fixed defense. This cannot automatically discover a new attack when the defense changes, adapting the attack requires a human to design another strategy. Our goal is to replace this fixed high-level strategy with a learned one: at round t, an attacker policy π_A receives an observation o_t^A drawn from its own clients' honest updates and produces an action a_t containing a client selection and an ordered weight-operation plan. The policy optimizes a verifiable reward through repeated interaction with the FL environment,

$$\max_{\pi_A} \mathbb{E}[R_{\text{att}}]. \tag{1}$$

The environment provides a fixed set of primitive operators and a fixed communication interface, but no fixed attack template or client-selection rule is supplied. We adopt a *partial-insider* threat model for this policy.

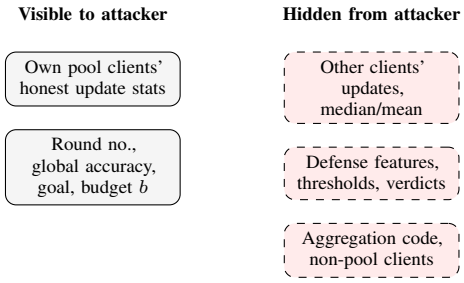

Fig. 1. Partial-insider observability boundary: what the attacker's policy can and cannot see, at both training and evaluation time.

Attacker capabilities. The attacker controls a fixed, controllable pool of $n_{\text{compromisable}}$ clients (5 or 10, out of $N = 20$, depending on the run) across the entire run. In any given round it may poison at most $b = \min(\text{budget}, n_{\text{compromisable}})$ of that pool, where the budget is drawn stochastically during training and fixed at evaluation time. For each pool client, the attacker observes per-layer statistics of that client's *own honest update* relative to the current global model (Section IV-B). It does not receive a robust reference such as the coordinate-wise median, another client's update, or a statistic computed over the pool. The attacker cannot alter the server's aggregation code or inject clients outside its fixed pool, and it cannot see any defense's internal state (features, thresholds, trust scores, or verdicts) at training or evaluation time.

Defense visibility. During training, the attacker's reward uses only global-model accuracy and a scalar confidence value returned by a training-time detector opponent (Section IV-C); its prompt never contains that detector's per-client features. At evaluation time, the frozen attacker's committed action is replayed without modification against every benchmark defense (Section V-E). The attacker never trains against, or adapts to, the internal state of any classical defense in the panel, so damage measured under those defenses reflects transfer rather than direct co-adaptation.

Out of scope. We do not model network-level attacks, Sybil attacks that fabricate new client identities [14], data-poisoning attacks mounted at a client's local training data, targeted/backdoor objectives, or collusion with a compromised aggregation server. These are natural extensions of the testbed (Section VII) but are not evaluated here.

Figure 1 summarizes this observability boundary.

IV. Attacker Design

A. Model and Data Environment

The environment simulates FL on MNIST with $N = 20$ clients under a non-IID partition using the FLTrust-style bias-q scheme, in which each client's local data is skewed toward a subset of classes with bias strength q [7]. The global model is a lightweight multilayer perceptron (MLP) with 970 parameters (`AvgPool2d(4)`

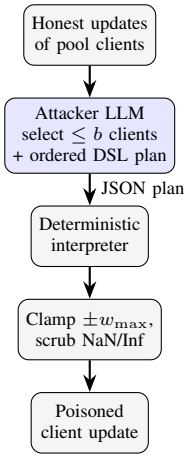

Fig. 2. Attacker action pipeline. The LLM never edits weights; it only emits a JSON plan of primitive operators that a fixed interpreter applies.

$\rightarrow$ `Linear(49,16)` $\rightarrow$ `ReLU` $\rightarrow$ `Linear(16,10)`). At this scale, per-layer statistics provide a compact observation an LLM can process as structured JSON rather than raw tensors, while the model still contains distinct weight and bias tensors the attacker can target selectively.

All N clients first train honestly via FedAvg for a fixed number of rounds; the resulting global model, per-client weights, and baseline accuracy are checkpointed and form the starting state for every attack round that follows. From that checkpoint, the attacker interacts round by round with the environment: it selects and poisons clients, the server aggregates, and the resulting accuracy and detector feedback (training only) drive the reward described in Section IV-C.

B. Attacker Contract

Input. The attacker observes the round number, current global accuracy, an attack goal (`untargeted_degrade`, evaluated in this paper, lower global test accuracy by a target amount), the controllable client pool P, and round budget b. For each pool client's honest update $\Delta = \text{local} - \text{global}$, the observation includes relative update norm $\|\Delta\|/\|G\|$, RMS delta, energy fraction, sign-flip fraction, standard-deviation ratio, absolute-mean ratio, and whole-model cosine similarity to the global model. These are normalized only against the global model; median, mean, and cross-client references remain unavailable to the partial insider. This representation is dimensionless, architecture independent, and unaffected by the round budget.

Output. A single JSON object selecting at most b pool clients and assigning each an ordered attack plan. Table I lists the ten primitive operators available in the DSL. Figure 2 shows the resulting pipeline: a deterministic interpreter (not the LLM) applies each plan in order to that client's benign weights, clamps the result to $\pm w_{\max}$, and scrubs any NaN/Inf values. Each operator applies to `"all"`, a named layer group (e.g. `"net.2"`), or a full parameter key (e.g. `"net.4.weight"`). Listing 1 shows the output schema.

C. Reward Design

$$R_{\text{att}} = \alpha \cdot \text{clip}\left(\frac{\Delta A}{\tau_a}, -0.5, 1.5\right) + \beta S - \gamma M - \delta C + \zeta D, \quad (2)$$

where $\Delta A = \text{acc}_{t-1} - \text{acc}_t$ is the induced accuracy drop and τ_a (`target_accuracy_drop`, default 0.20) is the target drop for the `untargeted_degrade` goal. $S \in [0, 1]$ is a continuous stealth term,

TABLE I: Attack operator DSL with 10 primitives. Each operator applies to `"all"`, a layer group, or a full parameter key.

Operator	Effect
`scale`	multiply targeted weights by a factor
`sign_flip`	negate the top fraction (by magnitude)
`add_gaussian_noise`	add $\mathcal{N}(0, \sigma^2)$ per coordinate
`mask`	zero a fraction (largest / smallest / random)
`clip`	clamp values to $\pm v$
`add_constant`	additive shift
`permute`	randomly shuffle coordinates
`scale_neurons`	scale a fraction of highest-norm output rows
`blend_random`	convex-blend with scaled random noise, α
`quantize`	round to a fixed step size

Listing 1: Illustrative attacker output schema (values are illustrative, not measured).

```
{
  "clients": [
    {
      "id": 3,
      "operations": [
        {"op": "sign_flip", "target": "net.4.weight",
         "fraction": 0.6},
        {"op": "scale", "target": "all", "factor": 1.4}
      ]
    }
  ]
}
```

the mean over the chosen clients of $1 - \hat{p}(\text{malicious})$, where $\hat{p}$ maps the training-time detector's confidence to a soft probability via $\hat{p} = 0.5 \pm 0.5 \cdot \text{confidence}$ (sign depending on its flag). M is the fraction of chosen clients whose plan was malformed or unparseable. $C = (n_{\text{used}} - 1)/(|P| - 1)$ is a use-fewer-clients penalty, zero for a single-client attack. $D \in [0, 1]$ is a diversity bonus, $D = 1 - \overline{\cos}$, the complement of the mean pairwise cosine similarity between the chosen clients' *perturbation vectors* $(w_i^{\text{poison}} - w_i^{\text{benign}})$, active only when $n_{\text{used}} > 1$. This rewards coordinated, distinct multi-client attacks over identical Sybil-like clones, which would otherwise present as an easy-to-catch, high-pairwise-cosine signal. Default weights, α=1.0, β=0.5, γ=1.0, δ=0.3, ζ=0.2 (Table III). The reward is continuous rather than binary because the GRPO advantage in Eq. (3) collapses to zero whenever every sampled rollout in a group receives the same reward, a failure mode particularly likely in the small, near-binary action space of choosing which clients to poison.

D. GRPO Training

Each attack round yields one exactly computable scalar reward, so a separate per-token value model offers limited benefit; GRPO samples G completions, scores them with the verifiable reward, and normalizes the scores within the group without a critic network [15]. For a group of G sampled completions $\{o_1, \ldots, o_G\}$ with rewards $\{r_1, \ldots, r_G\}$, the group-normalized advantage is

$$A_i = \frac{r_i - \text{mean}(r)}{\text{std}(r) + \epsilon}. \quad (3)$$

When $\text{std}(r) < \epsilon$, every A_i is set to 0 and the fraction of such groups is logged as a stalled-gradient signal. The policy is updated by

$$\mathcal{L} = \frac{1}{G} \sum_{i=1}^{G} \Big[- A_i \cdot \overline{\log \pi_\theta(o_i)} + \beta \cdot \overline{\text{KL}}(\pi_\theta \,\|\, \pi_{\text{ref}})_i \Big], \quad (4)$$

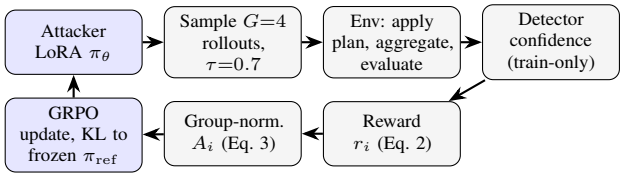

Fig. 3. GRPO training loop for the attacker policy. Only the attacker LoRA adapter is updated; the detector opponent supplies a stochastic confidence signal used solely to compute the reward.

where the bar denotes the mean over generated tokens, π_{ref} is the frozen base model (LoRA adapter disabled) evaluated with the k3 KL estimator, and β controls the KL penalty. Because updates are single-iteration, no PPO-style clipping term is required [16].

E. Stochastic Opponent Scoring and League Training

A critical failure mode in this setting is *zero-advantage collapse* (Figure 3). If the detector opponent responds deterministically, every sampled rollout of the attacker may face the same response; once the attacker is clearly winning or losing, all G rewards converge and $A_i \to 0$ in Eq. (3), eliminating the gradient. We therefore sample the frozen detector opponent at temperature $\tau = 0.7$ during scoring, so different attacker rollouts face varied responses and reward spread is restored. The committed action that advances the environment still uses the opponent's greedy response at $\tau = 0$, keeping the transition deterministic and reproducible. Training proceeds in phases: the attacker trains against a frozen opponent snapshot until it accumulates `success_streak` consecutive committed wins (default 3) or `max_phase_rounds` elapse without one (default 4000), after which the opponent snapshot is refreshed. With probability `league_prob` (default 0.25), the attacker instead faces a randomly selected past opponent snapshot from a stored league, which reduces overfitting to any single detector generation.

V. Experimental Setup

A. Implementation

The framework is implemented in PyTorch. LLM loading, LoRA injection, and GRPO training use the Unsloth library on top of Hugging Face PEFT and Transformers; no external RL trainer (e.g., TRL) dependency is required, since GRPO is implemented directly against the environment's verifiable reward.

B. Dataset and Partitioning

MNIST is partitioned across $N = 20$ clients using the FLTrust-style non-IID bias-q scheme [7] (default $q = 0.5$). The attacker's controllable pool is a strict minority in the small-budget run and half the federation in the large-budget run (Section VI-A).

C. Model and LLM Configuration

Table II summarizes the policy backbone. The attacker is a single LoRA adapter over a shared, frozen `Llama-3.2-3B-Instruct` base [18], [19]; a separate detector adapter over the same frozen base is used only as a training-time opponent (Section IV-E) and is not part of the evaluated attacker.

D. Training Hyperparameters

Table III lists the GRPO, schedule, and reward hyperparameters used in all runs unless otherwise noted.

TABLE II: Model and LLM configuration.

Component	Configuration
Global model	MLP, 970 parameters (MNIST)
LLM base	`Llama-3.2-3B-Instruct` (Unsloth), shared, frozen
Precision	bf16 LoRA
Evaluated policy	1× LoRA adapter (`attacker`)
Attention impl.	eager (default), sdpa (optional)

TABLE III: GRPO, schedule, and attacker-reward hyperparameters.

Hyperparameter	Value
Group size G	4
KL penalty β	0.02
Learning rate	1.0×10^{-5}
Opponent scoring temperature τ	0.7
Committed-action temperature	0.0 (greedy)
Success streak (opponent refresh)	3 consecutive wins
Max phase rounds (cap)	4000
Target accuracy drop τ_a	0.20 (train) / 0.10 (eval)
Reward weights $\alpha, \beta, \gamma, \delta, \zeta$	1.0, 0.5, 1.0, 0.3, 0.2
Non-IID bias q	0.5

E. Defense Panel

The trained attacker adapter is benchmarked, unmodified, against four independently evolving, fixed defenses drawn from the published literature; none of these defenses is proposed in this paper, and every one receives the exact same committed poisoned update set each round (Section V-F). Table IV summarizes the panel.

Multi-Krum and DnC require an assumed upper bound f/m on the number of malicious clients (a hyperparameter, not per-round ground truth); we default it to the evaluation poison budget, clamped to a benign majority, $f \leq \lfloor (N-1)/2 \rfloor$. FLTrust additionally requires a small clean root dataset (default 100 samples) that the server fine-tunes the global model on for one local epoch each round to obtain its trust reference.

F. Benchmark Protocol

Each benchmark round, the environment samples the controllable pool's honest updates, and the trained attacker LLM (temperature 0.7) produces *one* client selection and DSL plan against the reference (no-defense) accuracy. That single set of poisoned updates is then fed, unmodified, to every defense in the panel, each evolving its own global model and returning its own accept/reject verdicts (Figure 4). Holding the attack fixed while only the defense varies is what makes the resulting comparison attributable to the defense rather than to attack variance. We report, per defense, ground-truth detection metrics, true-positive rate (TPR/recall), false-positive rate (FPR), precision, and F1, and robustness metrics, final and mean test accuracy, mean accuracy drop versus the clean baseline, the fraction of rounds a poisoned client evaded detection (`thru`), and attack success rate (`succ`).

VI. Results and Discussion

The results in this section report two benchmark runs of the trained attacker adapter against the defense panel at two different poison budgets. They are single runs at single configurations, not yet averaged over seeds or intermediate budgets, and should be read as preliminary evidence of the mechanism's behavior rather than a fully replicated finding (Section VI-D).

TABLE IV: Defense panel. Every entry is a fixed, published mechanism used only as an evaluation target; none is proposed in this paper, and the attacker studied here never trains against, or adapts to, any of them.

Defense	Mechanism
FLTrust	Trust score $= \mathrm{ReLU}(\cos(\Delta_i, g_0))$ against a clean-root-set reference update g_0, surviving deltas rescaled to $\|g_0\|$ and trust-weighted averaged [7].
Multi-Krum	Keeps the $m{=}n{-}f$ clients with smallest summed squared distance to their $n{-}f{-}2$ closest neighbors [5].
DnC	Projects centered updates onto the top right singular vector of the client matrix, drops the $c \cdot m$ clients with largest squared projection [6].
DeFL	Tracks a per-layer Federated Gradient Norm Vector, detects Critical Learning Periods from its round-over-round rise, and votes a client malicious via an adaptive per-layer outlier threshold (MOUD) [9].

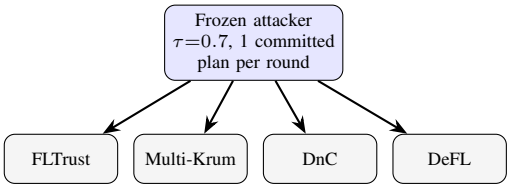

Fig. 4. Benchmark protocol: the same committed poisoned update set is replayed, unmodified, into four independently evolving, fixed defenses from the published literature.

A. Benchmark Run Configurations

Run A (small budget). $n = 50$ attack rounds, a clean baseline accuracy of 0.782, the `untargeted_degrade` goal with target accuracy drop $\tau_a = 0.10$, and an evaluation poison budget of $b = 2$ of a $n_{\text{compromisable}} = 5$-client pool, a strict minority consistent with Section III. Multi-Krum's and DnC's assumed-malicious-count hyperparameter was set to the true budget, $f = m = 2$.

Run B (large budget). $n = 20$ attack rounds, a clean baseline accuracy of 0.897, the same $\tau_a = 0.10$ goal, and a poison budget of $b = 10$ clients per round held as an exact quota, half of $N = 20$. This is deliberately *outside* the strict-minority regime: Byzantine-robust guarantees such as Multi-Krum's own bound, $n \geq 2f + 3$, generally require the malicious fraction to stay strictly below $1/2$, so Run B is a boundary stress test of what happens once that assumption is no longer satisfied. Because Multi-Krum's and DnC's assumed-malicious-count hyperparameter is clamped to $\lfloor (N-1)/2 \rfloor = 9$, both were handed $f = m = 9$ against a true budget of 10, a one-client underestimate that arises mechanically from the clamp itself. In both runs the attacker adapter was frozen and sampled at temperature 0.7; every defense in the panel evolved its own global model against the identical committed attack each round.

B. The Small-Budget Regime

At b=2 of 5, a strict minority of the federation, Multi-Krum achieves near-perfect results, 100% detection, 0% FPR, 0.784 final accuracy, 0% attack success, despite never having seen this attacker during training: the attacker's committed perturbations were, every round, distant enough from the honest bulk under Multi-Krum's nearest-neighbor distance score to be excluded outright. FLTrust and DnC show a second pattern, moderate binary detection (75% and 66%) at high or moderate FPR, yet negligible realized damage ($+0.004$ and $+0.042$ mean accuracy drop), because both apply a continuous re-weighting or projection to every surviving update rather than a hard accept/reject filter, so even rounds where a poisoned client's flag is missed contribute little to the aggregate. DeFL inverts this pattern: the *highest* detection rate of any defense in the panel (91%) and the best F1 (0.71), yet the most damage (final accuracy 0.338, 56% attack success), because its Critical-Learning-Period gating makes its filtering strength non-uniform round to round, and a handful of missed early rounds leave the global model in a degraded state that later, correctly-flagged rounds cannot undo. Per-round detection metrics alone did not predict which defense actually protected the model.

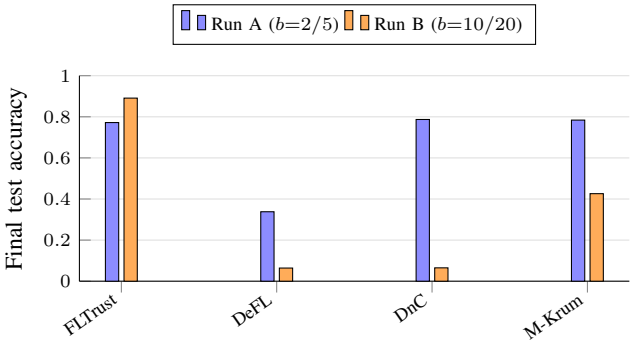

Fig. 5. Final test accuracy per defense under the identical, frozen attacker at each budget. Run A baseline 0.782; Run B baseline 0.897 (dashed references omitted for clarity).

C. The Large-Budget Regime

At b=10 of 20, half the federation, the picture changes sharply and unevenly (Table VI, Figure 5). DeFL's detection rate falls from Run A's best-in-panel 91% to exactly 0%, and its final accuracy (0.064), mean drop ($+0.825$), and attack success (100%) show a near-total collapse; DnC degrades almost as completely (41.5% detect, 48.5% FPR, essentially a coin flip, 100% attack success). Multi-Krum holds up only partially: its mean accuracy (0.537) is the least damaged of the three collapsing defenses, but a 0.36 absolute drop and 95.9% attack success remain. We read this as the panel failing close to where its own theory says it should rather than as a flaw the attack specifically exploited: Multi-Krum's bound $n \geq 2f + 3$ is violated outright at $f = 10$, $n = 20$, and both Multi-Krum's and DnC's assumed-malicious-count hyperparameter is mechanically clamped to $f = 9$, one client short of the truth, because $\lfloor (N-1)/2 \rfloor = 9$ is the largest value the clamp permits at $N = 20$.

What did *not* collapse is the more surprising result. FLTrust (0.891 final accuracy, $+0.006$ drop) remains close to the clean baseline even with half the federation compromised. Unlike Multi-Krum and DnC, FLTrust does not depend on an assumed malicious-count hyperparameter: it anchors every update to a clean root-set reference independent of how many clients are compromised. This suggests that robustness at large poison budgets tracks whether a defense's assumptions scale with the malicious fraction, not merely its detection accuracy at a single, smaller budget.

D. Limitations

These results come from two single runs at two fixed configurations. First, neither run has been repeated across seeds or at intermediate budgets between b=2 and b=10, so the shape of the transition

TABLE V: Run A defense-panel results for 50 attack rounds, baseline accuracy 0.782, target drop $\tau_a = 0.10$, and poison budget $b = 2$ of 5.

Defense	Detect%	FPR	Prec	F1	Final acc.	Mean acc.	Acc. drop	thru	succ
FLTrust	75.0%	71.7%	0.10	0.18	0.772	0.778	+0.004	46.0%	0.0%
DeFL	91.0%	7.3%	0.58	0.71	0.338	0.451	+0.332	18.0%	56.0%
DnC	66.0%	3.8%	0.66	0.66	0.787	0.741	+0.042	68.0%	6.0%
Multi-Krum	100.0%	0.0%	1.00	1.00	0.784	0.784	−0.001	0.0%	0.0%

TABLE VI: Run B defense-panel results for 20 attack rounds, baseline accuracy 0.897, target drop $\tau_a = 0.10$, and poison budget $b = 10$ (exact quota).

Defense	Detect%	FPR	Prec	F1	Final acc.	Mean acc.	Acc. drop	thru	succ
FLTrust	95.0%	77.0%	0.55	0.70	0.891	0.891	+0.006	5.0%	5.7%
DeFL	1.0%	2.0%	0.33	0.02	0.064	0.072	+0.825	100.0%	100.0%
DnC	41.5%	48.5%	0.46	0.44	0.065	0.054	+0.843	100.0%	100.0%
Multi-Krum	53.5%	36.5%	0.59	0.56	0.426	0.537	+0.359	100.0%	95.9%

between the two regimes is unknown. Second, Run B could not isolate the mismatched-assumption question (the one-client clamp underestimate) from the budget-exceeds-guarantee question for Multi-Krum and DnC; a deliberate ablation holding the true budget fixed while varying only the assumed count is still needed. Third, the divergence between detection metrics and realized protection observed for DeFL is diagnosed here from the mechanism's design, not from an ablation isolating its Critical-Learning-Period trigger. Fourth, the benchmark harness runs no honest-FL recovery interlude between attack rounds, so any defense that misses even a small number of early rounds accumulates damage for the remainder of the run, which stresses worst-case persistence and may overstate real-world damage relative to a deployment that periodically re-trains or resets. Fifth, both runs operate over a compact, 970-parameter MLP; per-layer statistics of this size may not transfer to substantially larger architectures without redesigning the feature extractor and the DSL's targeting granularity.

VII. Conclusion

We presented a zero-touch, LLM-directed FL attacker that learns, via GRPO over a verifiable reward, which clients to poison and how to compose primitive weight operators against them, without a hand-authored attack template. Benchmarked unmodified against four independently evolving, fixed defenses from the published literature across two poison-budget regimes, the attacker was fully neutralized by Multi-Krum and FLTrust at a strict-minority budget, and drove DeFL and DnC toward near-total success once its budget reached half the federation, while FLTrust held the global model close to baseline at that same larger budget. Per-round detection metrics did not reliably predict realized protection. Future work will replicate both runs across seeds, sweep intermediate budgets, run a mismatched-assumption ablation for Multi-Krum/DnC, extend the global model beyond a single compact MLP, and study Sybil client creation and targeted (backdoor) attack goals against the same panel.

References

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, 2017, pp. 1273–1282.

[2] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *Proc. 23rd Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, 2020, pp. 2938–2948.

[3] A. N. Bhagoji, S. Chakraborty, P. Mittal, and S. Calo, "Analyzing federated learning through an adversarial lens," in *Proc. 36th Int. Conf. Machine Learning (ICML)*, 2019, pp. 634–643.

[4] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. 35th Int. Conf. Machine Learning (ICML)*, 2018, pp. 5650–5659.

[5] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[6] V. Shejwalkar and A. Houmansadr, "Manipulating the Byzantine: Optimizing model poisoning attacks and defenses for federated learning," in *Proc. Network and Distributed System Security Symp. (NDSS)*, 2021.

[7] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "FLTrust: Byzantine-robust federated learning via trust bootstrapping," in *Proc. Network and Distributed System Security Symp. (NDSS)*, 2021.

[8] C. Fung, C. J. M. Yoon, and I. Beschastnikh, "The limitations of federated learning in sybil settings," in *Proc. 23rd Int. Symp. Research in Attacks, Intrusions and Defenses (RAID)*, 2020, pp. 301–316.

[9] G. Yan *et al.*, "DeFL: Defending against model poisoning attacks in federated learning via critical learning periods awareness," in *Proc. AAAI Conf. Artificial Intelligence*, 2023.

[10] M. Fang, X. Cao, J. Jia, and N. Z. Gong, "Local model poisoning attacks to Byzantine-robust federated learning," in *Proc. 29th USENIX Security Symp.*, 2020, pp. 1605–1622.

[11] C. Xie, O. Koyejo, and I. Gupta, "Fall of empires: Breaking Byzantine-tolerant SGD by inner product manipulation," in *Proc. 36th Conf. Uncertainty in Artificial Intelligence (UAI)*, 2020, pp. 261–270.

[12] G. Baruch, M. Baruch, and Y. Goldberg, "A little is enough: Circumventing defenses for distributed learning," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[13] H. Wang, K. Sreenivasan, S. Rajput, H. Vishwakarma, S. Agarwal, J.-y. Sohn, K. Lee, and D. Papailiopoulos, "Attack of the tails: Yes, you really can backdoor federated learning," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 16070–16080.

[14] X. Cao and N. Z. Gong, "MPAF: Model poisoning attacks to federated learning based on fake clients," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2022, pp. 3396–3404.

[15] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," 2024, arXiv:2402.03300.

[16] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, arXiv:1707.06347.

[17] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 27730–27744.

[18] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.

[19] A. Dubey *et al.*, "The Llama 3 herd of models," 2024, arXiv:2407.21783.

[20] T. D. Nguyen *et al.*, "FLAME: Taming backdoors in federated learning," in *Proc. 31st USENIX Security Symp.*, 2022, pp. 1415–1432.